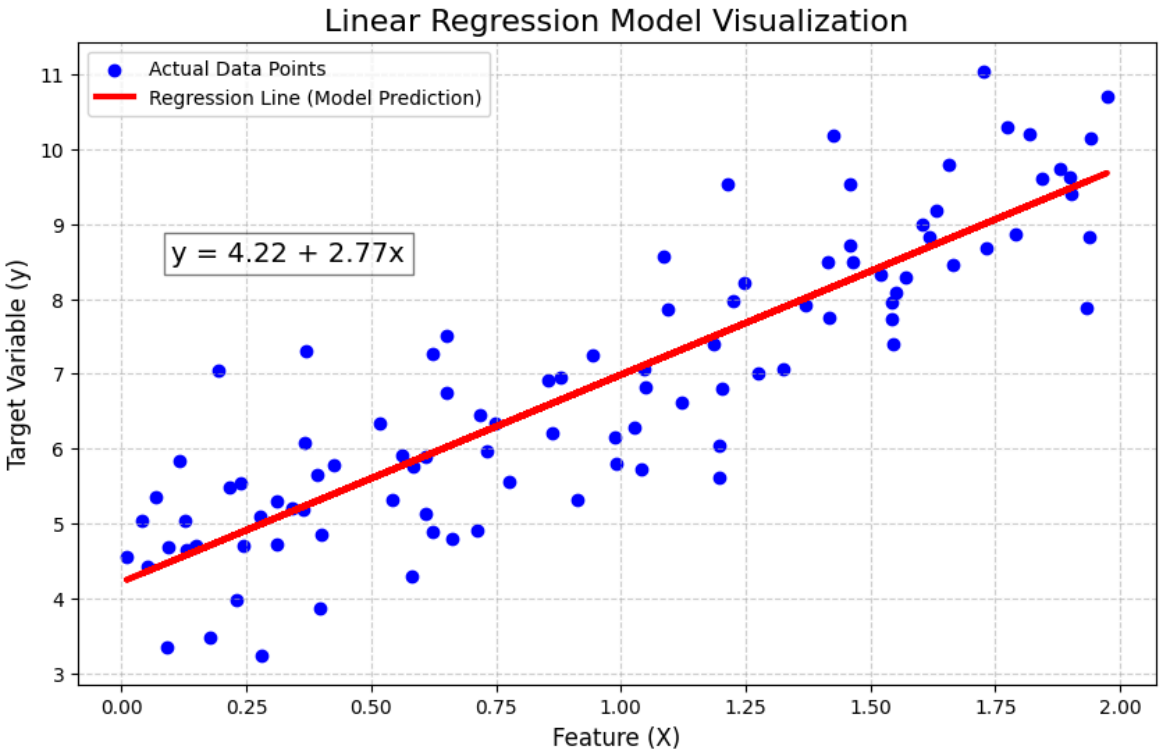
A Comprehensive Guide to Machine Learning Model Visualizations

Linear Regression Chart Code (Matplotlib)

In [1]:
```
import matplotlib.pyplot as plt
import numpy as np
from sklearn.linear_model import LinearRegression

# 1. Create dummy data
np.random.seed(42)
X = 2 * np.random.rand(100, 1)
y = 4 + 3 * X + np.random.randn(100, 1)

# 2. Fit the Linear Regression model
model = LinearRegression()
model.fit(X, y)
y_pred = model.predict(X)

# 3. Create the plot
plt.figure(figsize=(10, 6))

# Plot the original data points
plt.scatter(X, y, color='blue', label='Actual Data Points')

# Plot the regression line
plt.plot(X, y_pred, color='red', linewidth=3, label='Regression Line (Model Pred

# Plot residuals as vertical lines
# for x_i, y_i, y_pred_i in zip(X, y, y_pred):
#     plt.plot([x_i, x_i], [y_i, y_pred_i], 'g--', linewidth=0.5)

# Add title and labels
plt.title('Linear Regression Model Visualization', fontsize=16)
plt.xlabel('Feature (X)', fontsize=12)
plt.ylabel('Target Variable (y)', fontsize=12)
plt.legend()
plt.grid(True, linestyle='--', alpha=0.6)

# Add text for the equation
equation = f'y = {model.intercept_[0]:.2f} + {model.coef_[0][0]:.2f}x'
plt.text(0.1, 8.5, equation, fontsize=14, bbox=dict(facecolor='white', alpha=0.5

plt.show()
```

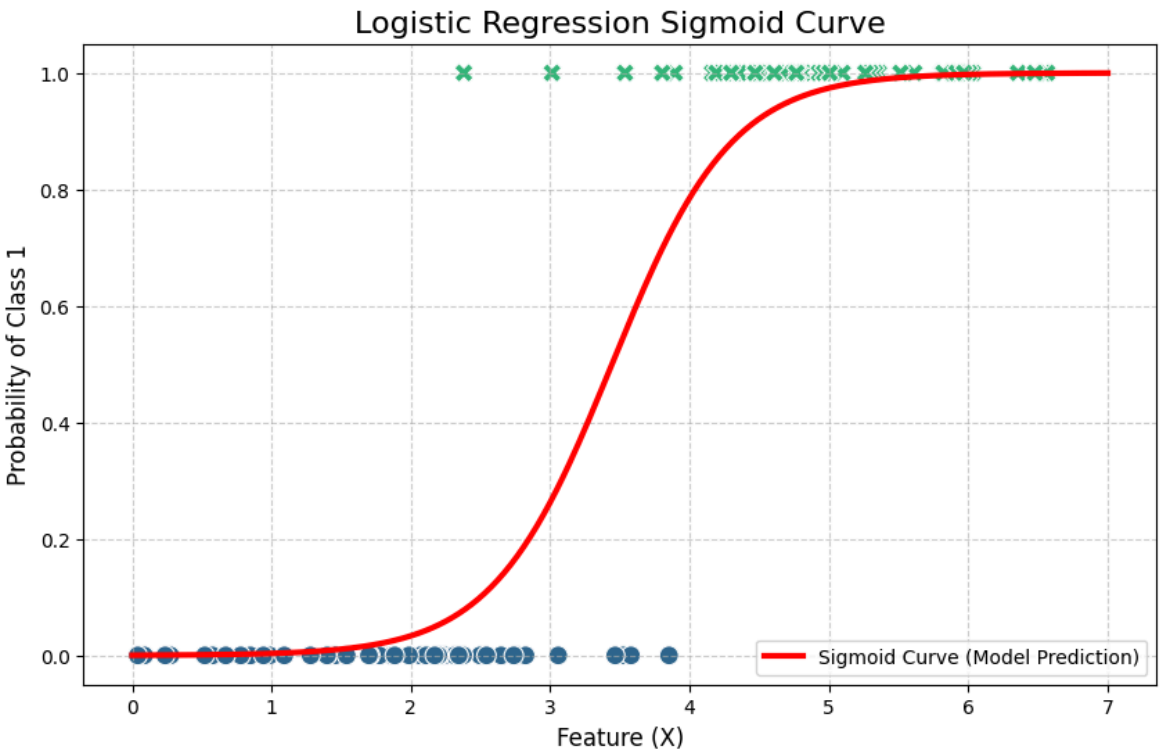


Logistic Regression Chart Code (Seaborn & Matplotlib)

In [2]:

```
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
from sklearn.linear_model import LogisticRegression

# 1. Create dummy data (two distinct clusters)
np.random.seed(42)
X1 = np.random.normal(2, 1, 50)
X2 = np.random.normal(5, 1, 50)
X = np.concatenate((X1, X2)).reshape(-1, 1)
y = np.array([0] * 50 + [1] * 50)

# 2. Fit the Logistic Regression model
model = LogisticRegression()
model.fit(X, y)

# 3. Create the plot
plt.figure(figsize=(10, 6))

# Plot the data points
sns.scatterplot(x=X.flatten(), y=y, hue=y, palette='viridis', style=y, s=100, le

# Create the sigmoid curve
X_test = np.linspace(0, 7, 300).reshape(-1, 1)
y_proba = model.predict_proba(X_test)[:, 1] # Probability of class 1

plt.plot(X_test, y_proba, color='red', linewidth=3, label='Sigmoid Curve (Model

# Add title and labels
plt.title('Logistic Regression Sigmoid Curve', fontsize=16)
```

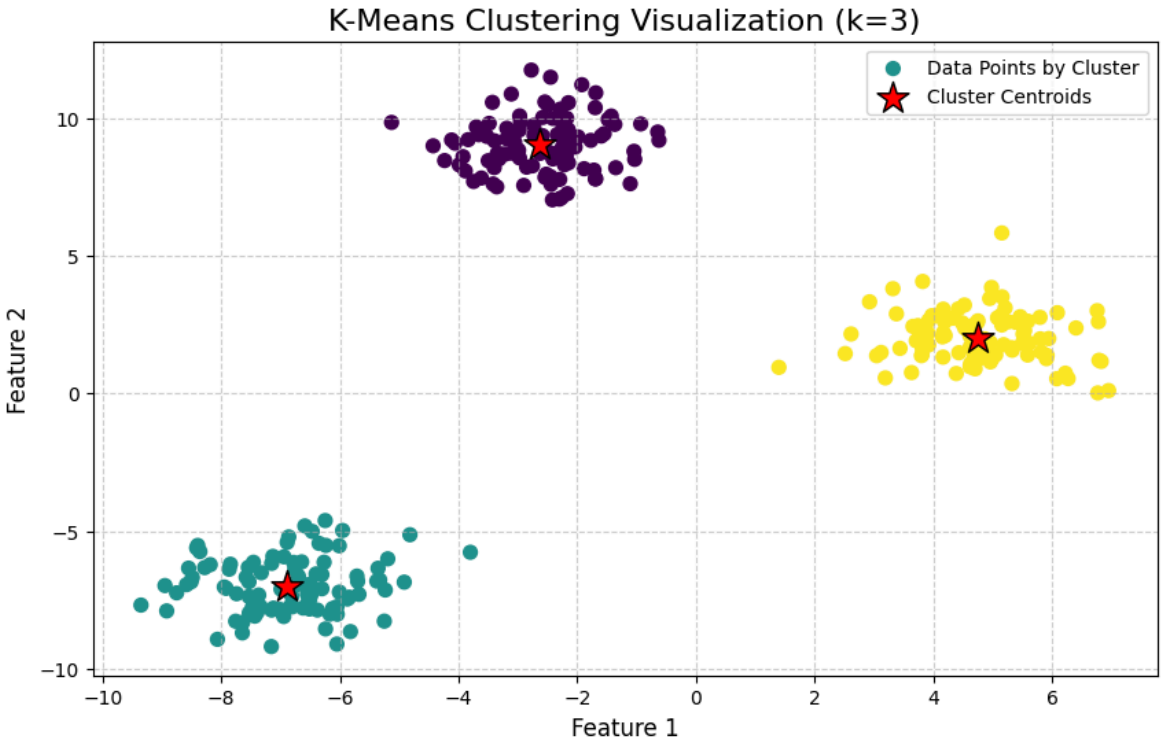
```
plt.xlabel('Feature (X)', fontsize=12)
plt.ylabel('Probability of Class 1', fontsize=12)
plt.grid(True, linestyle='--', alpha=0.6)
plt.legend()

plt.show()
```



K-Means Clustering Chart Code (Matplotlib)

In [3]:
```
import matplotlib.pyplot as plt
import numpy as np
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

# 1. Create dummy data with 3 distinct blobs
X, y_true = make_blobs(n_samples=300, centers=3, cluster_std=1.0, random_state=4

# 2. Fit the K-Means model
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
kmeans.fit(X)
y_kmeans = kmeans.predict(X)
centroids = kmeans.cluster_centers_

# 3. Create the plot
plt.figure(figsize=(10, 6))

# Plot the data points, colored by their cluster
plt.scatter(X[:, 0], X[:, 1], c=y_kmeans, s=50, cmap='viridis', label='Data Poin

# Plot the cluster centroids
plt.scatter(centroids[:, 0], centroids[:, 1], marker='*', s=300,
            c='red', edgecolors='black', label='Cluster Centroids')
```

```
# Add title and labels
plt.title('K-Means Clustering Visualization (k=3)', fontsize=16)
plt.xlabel('Feature 1', fontsize=12)
plt.ylabel('Feature 2', fontsize=12)
plt.legend()
plt.grid(True, linestyle='--', alpha=0.6)

plt.show()
```

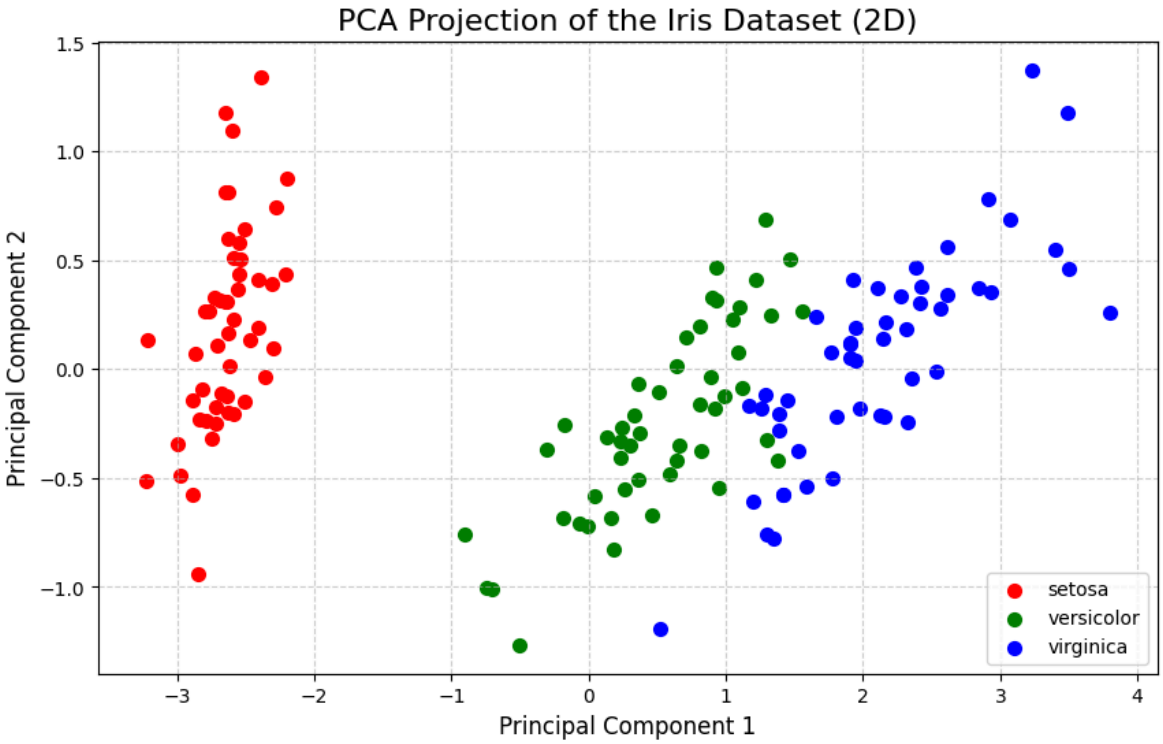


PCA Projection Chart Code (Matplotlib)

In [4]:

```
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.datasets import load_iris

# 1. Load the Iris dataset (4 features, 3 classes)
iris = load_iris()
X = iris.data
y = iris.target
target_names = iris.target_names

# 2. Apply PCA to reduce dimensions to 2
pca = PCA(n_components=2)
X_reduced = pca.fit_transform(X)

# 3. Create the plot
plt.figure(figsize=(10, 6))

# Plot the reduced data points, colored by their class
colors = ['red', 'green', 'blue']
for color, i, target_name in zip(colors, [0, 1, 2], target_names):
    plt.scatter(X_reduced[y == i, 0], X_reduced[y == i, 1],
                color=color, lw=2, label=target_name)

# Add title and labels
```

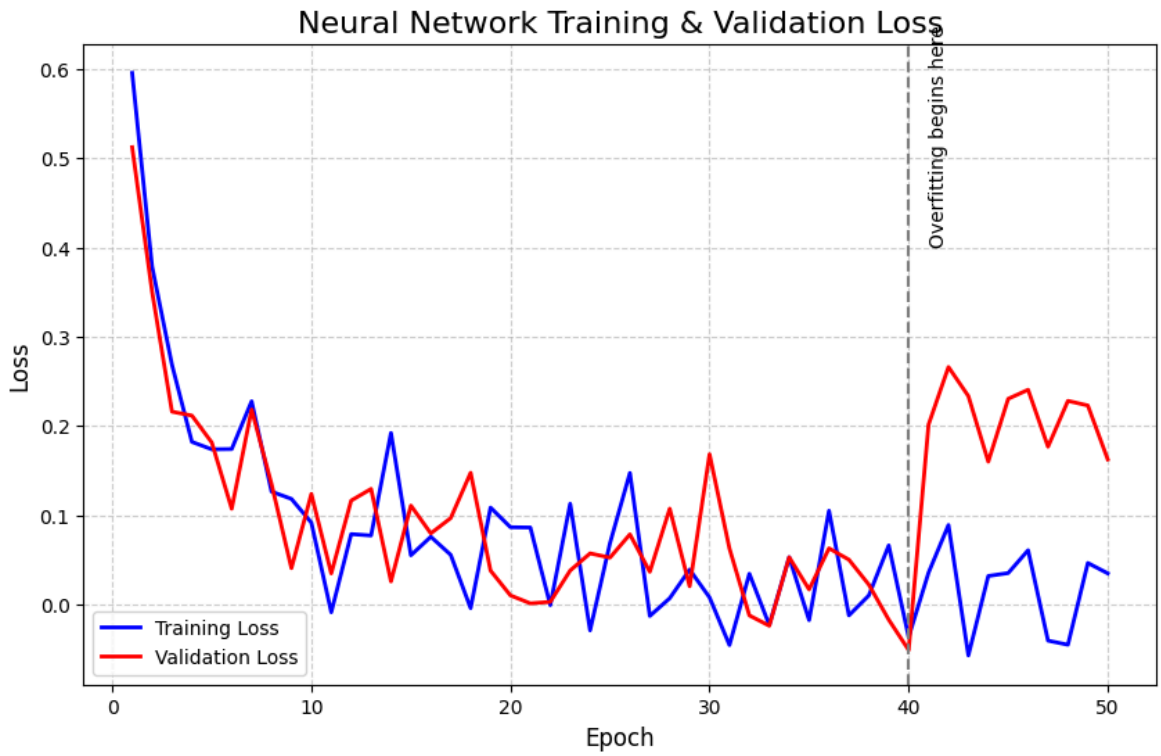
```
plt.title('PCA Projection of the Iris Dataset (2D)', fontsize=16)
plt.xlabel('Principal Component 1', fontsize=12)
plt.ylabel('Principal Component 2', fontsize=12)
plt.legend(loc='best', shadow=False, scatterpoints=1)
plt.grid(True, linestyle='--', alpha=0.6)

plt.show()
```



Neural Network Loss Plot Code (Matplotlib)

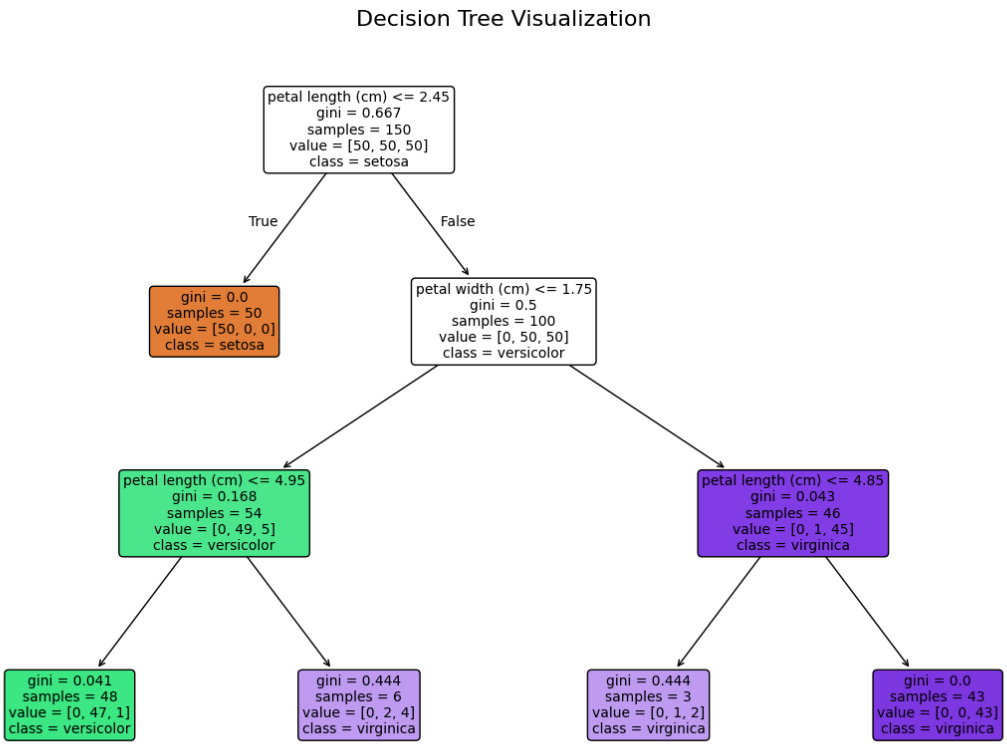
In [5]:
```
import matplotlib.pyplot as plt
import numpy as np

# 1. Simulate training and validation loss data
epochs = np.arange(1, 51)
training_loss = 1.0 / (epochs + 0.5) + np.random.normal(0, 0.05, 50)
validation_loss = 1.0 / (epochs + 1) + np.random.normal(0, 0.05, 50)
validation_loss[40:] += 0.2 # Simulate overfitting

# 2. Create the plot
plt.figure(figsize=(10, 6))

# Plot the loss curves
plt.plot(epochs, training_loss, label='Training Loss', color='blue', linewidth=2
plt.plot(epochs, validation_loss, label='Validation Loss', color='red', linewidt

# Add title and labels
plt.title('Neural Network Training & Validation Loss', fontsize=16)
plt.xlabel('Epoch', fontsize=12)
plt.ylabel('Loss', fontsize=12)
plt.legend()
plt.grid(True, linestyle='--', alpha=0.6)
```

```
# Add a dashed line to indicate potential overfitting
plt.axvline(x=40, color='gray', linestyle='--', linewidth=1.5, label='Potential
plt.text(41, 0.4, 'Overfitting begins here', rotation=90, va='bottom')

plt.show()
```



Decision Tree Chart Code (Matplotlib)

In [6]:
```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier, plot_tree

# 1. Load data
iris = load_iris()
X = iris.data
y = iris.target

# 2. Fit the model
tree_clf = DecisionTreeClassifier(max_depth=3, random_state=42)
tree_clf.fit(X, y)

# 3. Plot the decision tree
plt.figure(figsize=(15, 10))
plot_tree(tree_clf,
          feature_names=iris.feature_names,
          class_names=iris.target_names,
          filled=True,
          rounded=True,
          fontsize=10)

plt.title('Decision Tree Visualization', fontsize=16)
plt.show()
```

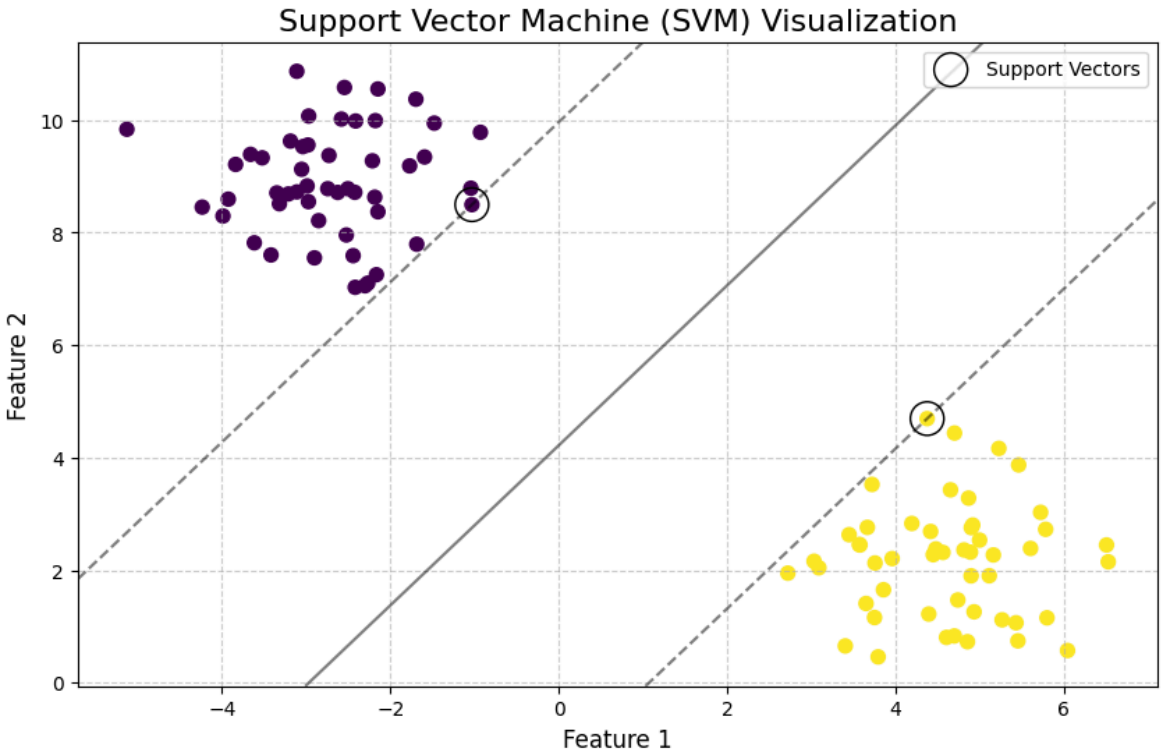


Support Vector Machine (SVM) Chart Code (Matplotlib)

In [7]:

```
import matplotlib.pyplot as plt
import numpy as np
from sklearn.svm import SVC
from sklearn.datasets import make_blobs

# 1. Create dummy data
X, y = make_blobs(n_samples=100, centers=2, random_state=42)

# 2. Fit the SVM model
model = SVC(kernel='linear', C=1.0)
model.fit(X, y)

# 3. Create the plot
plt.figure(figsize=(10, 6))

# Plot data points
plt.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='viridis')

# Create the decision function boundary lines
ax = plt.gca()
xlim = ax.get_xlim()
ylim = ax.get_ylim()

# Create grid to evaluate model
xx = np.linspace(xlim[0], xlim[1], 30)
yy = np.linspace(ylim[0], ylim[1], 30)
YY, XX = np.meshgrid(yy, xx)
xy = np.vstack([XX.ravel(), YY.ravel()]).T
Z = model.decision_function(xy).reshape(XX.shape)
```

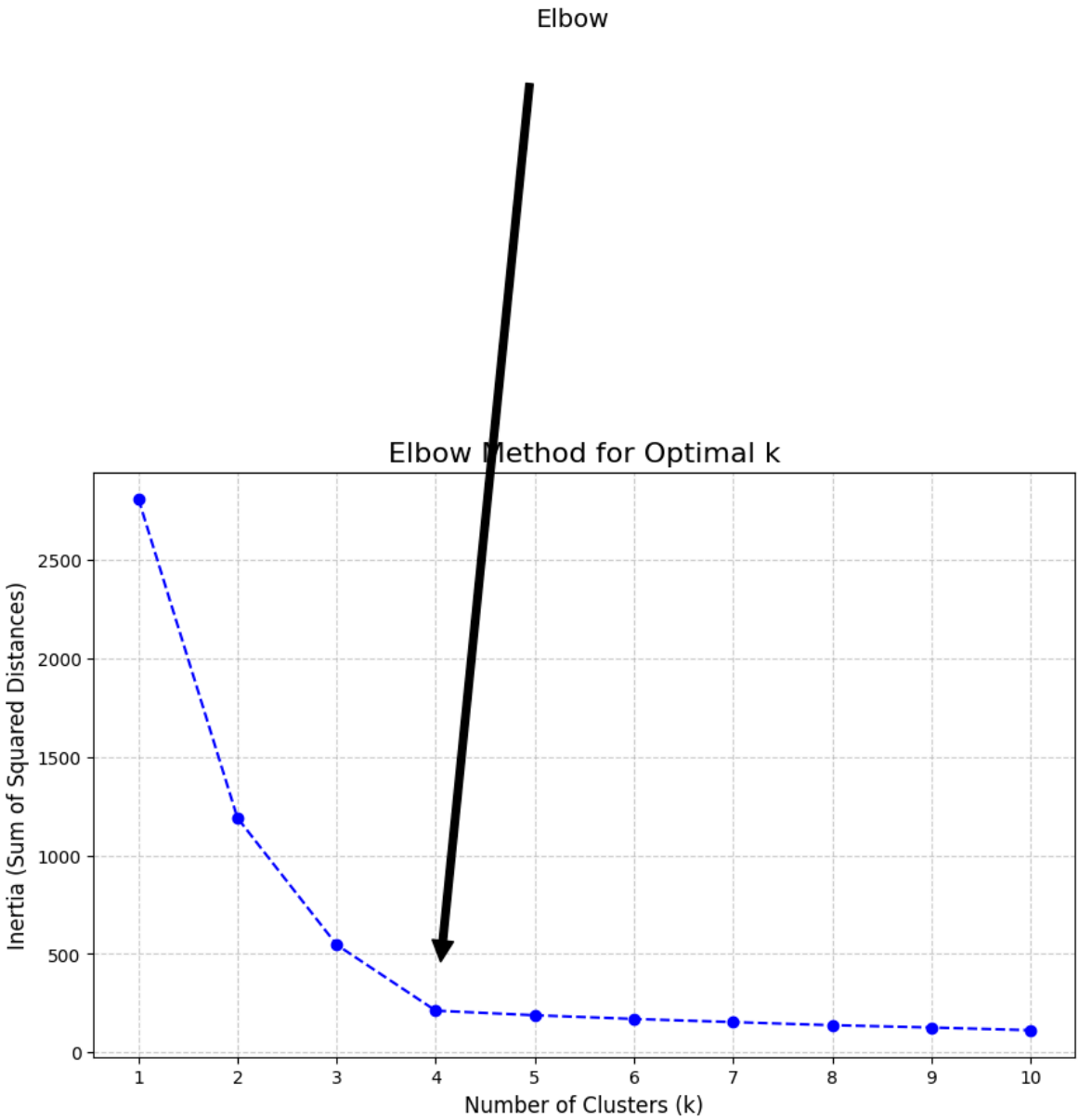
```
# Plot decision boundary and margins
ax.contour(XX, YY, Z, colors='k', levels=[-1, 0, 1], alpha=0.5,
           linestyles=['--', '-', '--'])

# Plot support vectors
ax.scatter(model.support_vectors_[:, 0], model.support_vectors_[:, 1], s=300,
           linewidth=1, facecolors='none', edgecolors='k', label='Support Vector

plt.title('Support Vector Machine (SVM) Visualization', fontsize=16)
plt.xlabel('Feature 1', fontsize=12)
plt.ylabel('Feature 2', fontsize=12)
plt.legend()
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```



K-Means Elbow Method Chart Code (Matplotlib)

In [8]:
```
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

# 1. Create dummy data
X, _ = make_blobs(n_samples=300, centers=4, cluster_std=0.60, random_state=0)

# 2. Calculate the inertia for a range of k values
inertia = []
for i in range(1, 11):
    kmeans = KMeans(n_clusters=i, random_state=42, n_init=10)
    kmeans.fit(X)
    inertia.append(kmeans.inertia_)

# 3. Plot the elbow curve
```

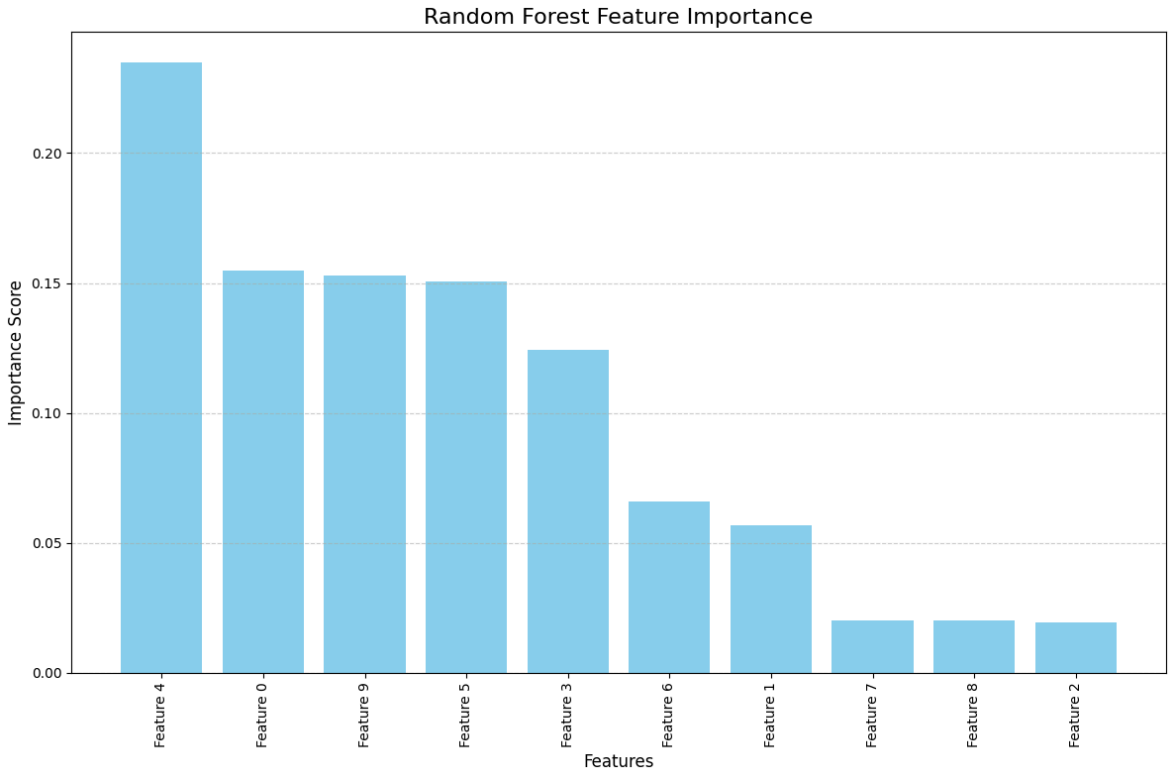
```python
plt.figure(figsize=(10, 6))
plt.plot(range(1, 11), inertia, marker='o', linestyle='--', color='b')

# Add title and labels
plt.title('Elbow Method for Optimal k', fontsize=16)
plt.xlabel('Number of Clusters (k)', fontsize=12)
plt.ylabel('Inertia (Sum of Squared Distances)', fontsize=12)
plt.xticks(range(1, 11))
plt.grid(True, linestyle='--', alpha=0.6)

# Highlight the "elbow" point (often an inflection point)
plt.annotate('Elbow', xy=(4, inertia[3]), xytext=(5, inertia[3] + 5000),
             arrowprops=dict(facecolor='black', shrink=0.05),
             fontsize=14)

plt.show()
```



Random Forest Feature Importance Chart Code (Matplotlib)

In [9]:

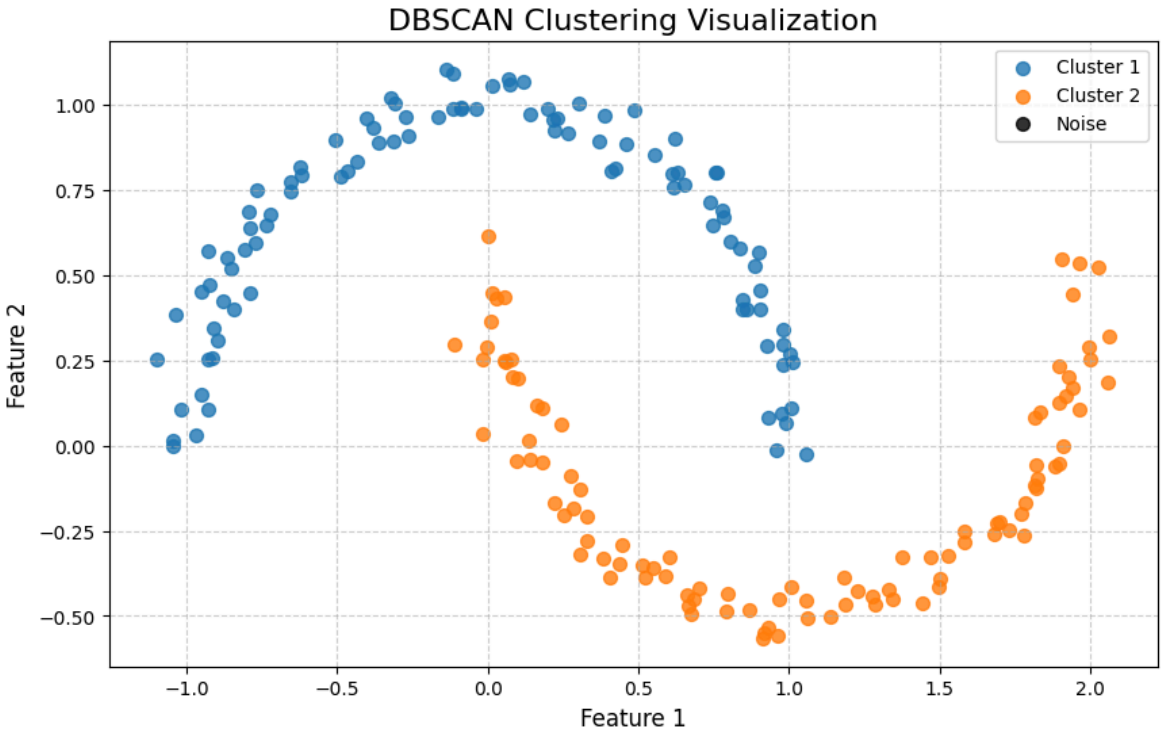
```python
import matplotlib.pyplot as plt
import numpy as np
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import make_classification

# 1. Create dummy data with 5 features
X, y = make_classification(n_samples=1000, n_features=10, n_informative=5,
                           n_redundant=2, n_classes=2, random_state=42)
feature_names = [f'Feature {i}' for i in range(10)]

# 2. Fit the Random Forest model
rf_clf = RandomForestClassifier(n_estimators=100, random_state=42)
rf_clf.fit(X, y)

# 3. Get feature importances and sort them
importances = rf_clf.feature_importances_
indices = np.argsort(importances)[::-1]
sorted_importances = importances[indices]
sorted_feature_names = [feature_names[i] for i in indices]

# 4. Create the bar chart
plt.figure(figsize=(12, 8))
plt.bar(range(X.shape[1]), sorted_importances, color='skyblue', align='center')
plt.xticks(range(X.shape[1]), sorted_feature_names, rotation=90)

# Add title and labels
plt.title('Random Forest Feature Importance', fontsize=16)
plt.xlabel('Features', fontsize=12)
plt.ylabel('Importance Score', fontsize=12)
plt.grid(axis='y', linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()
```



DBSCAN Clustering Chart Code (Matplotlib)

In [10]:

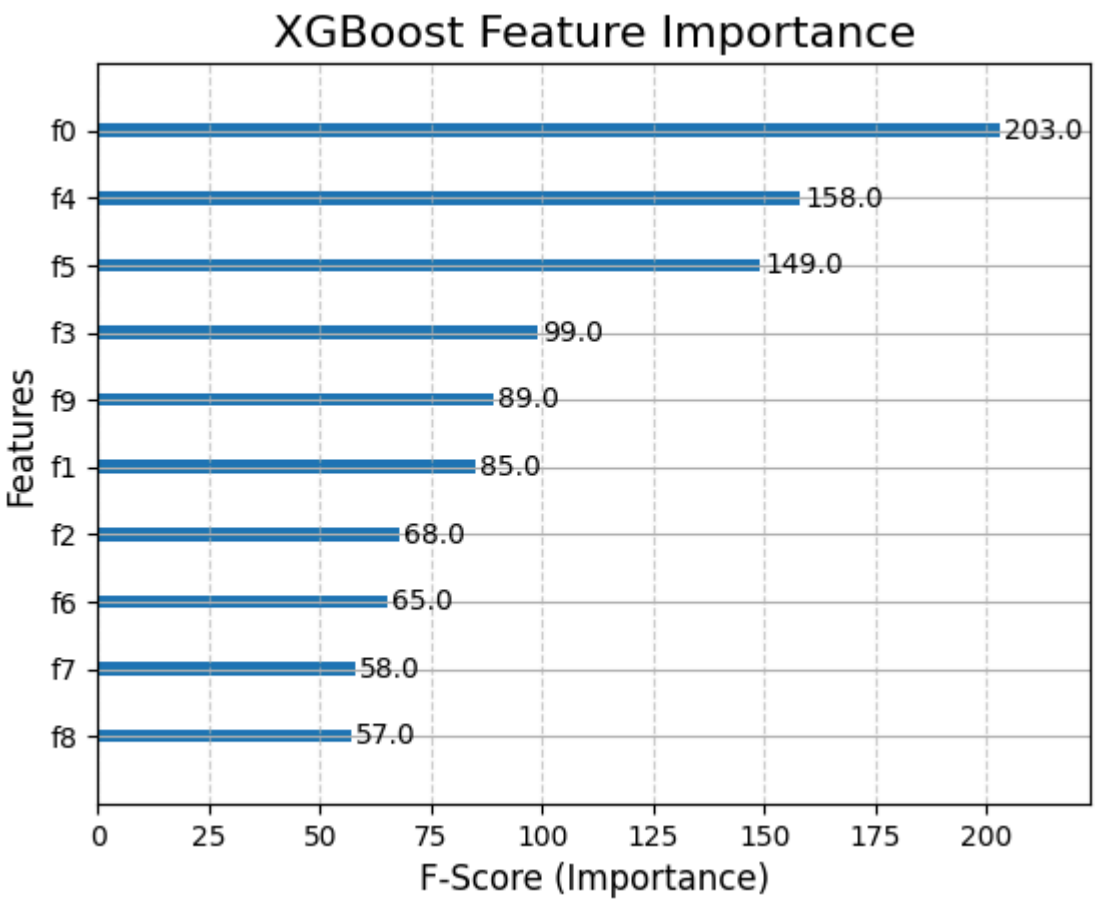
```
import matplotlib.pyplot as plt
import numpy as np
from sklearn.cluster import DBSCAN
from sklearn.datasets import make_moons

# 1. Create dummy data with a non-linear shape
X, _ = make_moons(n_samples=200, noise=0.05, random_state=42)

# 2. Fit the DBSCAN model
dbscan = DBSCAN(eps=0.3, min_samples=5)
clusters = dbscan.fit_predict(X)

# 3. Create the plot
plt.figure(figsize=(10, 6))

# Plot data points
plt.scatter(X[clusters == 0, 0], X[clusters == 0, 1], label='Cluster 1', s=50, a
plt.scatter(X[clusters == 1, 0], X[clusters == 1, 1], label='Cluster 2', s=50, a
plt.scatter(X[clusters == -1, 0], X[clusters == -1, 1], label='Noise', s=50, alp

# Add title and labels
plt.title('DBSCAN Clustering Visualization', fontsize=16)
plt.xlabel('Feature 1', fontsize=12)
plt.ylabel('Feature 2', fontsize=12)
plt.legend()
plt.grid(True, linestyle='--', alpha=0.6)

plt.show()
```



XGBoost Feature Importance Chart Code (Matplotlib)

In [13]:
```
import matplotlib.pyplot as plt
import numpy as np
import xgboost as xgb
from sklearn.datasets import make_classification
from xgboost import plot_importance

# 1. Create dummy data with 10 features
X, y = make_classification(n_samples=1000, n_features=10, n_informative=5,
                           n_redundant=2, n_classes=2, random_state=42)

# 2. Fit the XGBoost model (without use_label_encoder)
model = xgb.XGBClassifier(n_estimators=100, random_state=42, eval_metric='loglos
model.fit(X, y)

# 3. Create the feature importance plot
plt.figure(figsize=(12, 8))
plot_importance(model)

# Add title and labels
plt.title('XGBoost Feature Importance', fontsize=16)
plt.xlabel('F-Score (Importance)', fontsize=12)
plt.ylabel('Features', fontsize=12)
plt.grid(axis='x', linestyle='--', alpha=0.6)

plt.show()
```

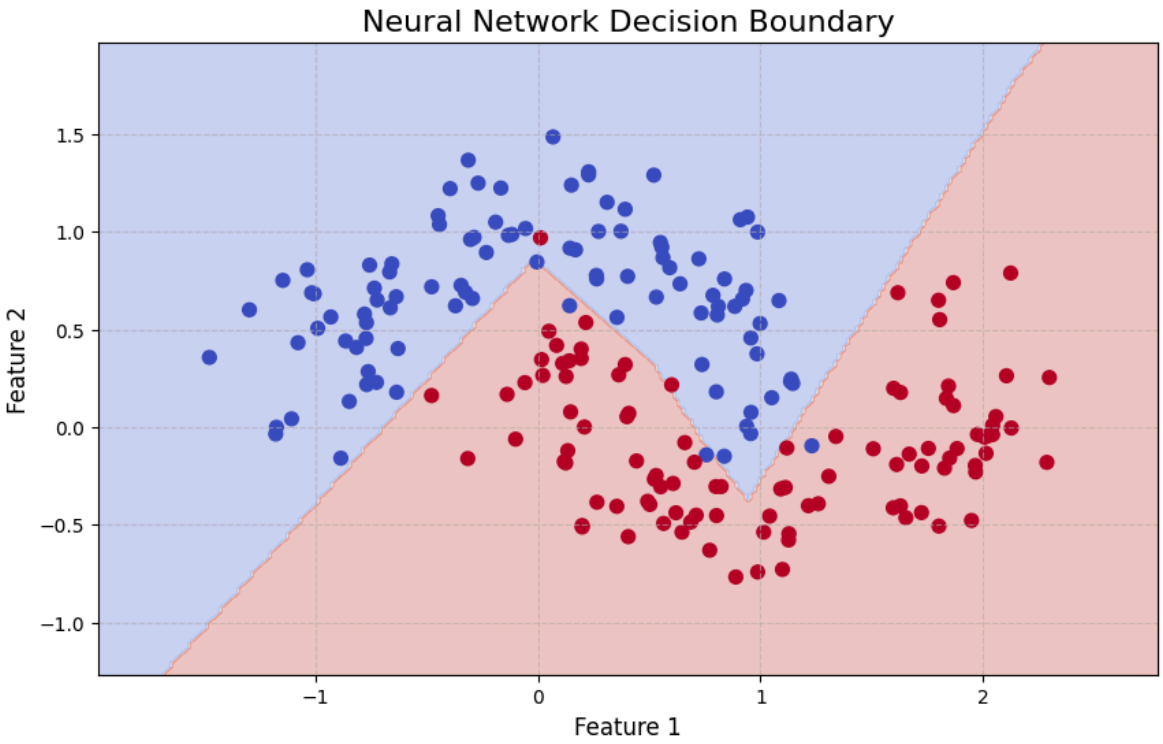
```
<Figure size 1200x800 with 0 Axes>
```



Neural Network Decision Boundary Chart Code (Matplotlib)

In [13]:
```
import matplotlib.pyplot as plt
import numpy as np
from sklearn.neural_network import MLPClassifier
from sklearn.datasets import make_moons

# 1. Create dummy data
X, y = make_moons(n_samples=200, noise=0.2, random_state=42)

# 2. Fit the Neural Network model
mlp = MLPClassifier(solver='lbfgs', hidden_layer_sizes=(10,),
                    random_state=42, max_iter=1000)
mlp.fit(X, y)

# 3. Create the plot
plt.figure(figsize=(10, 6))

# Define the boundaries for the plot
x_min, x_max = X[:, 0].min() - .5, X[:, 0].max() + .5
y_min, y_max = X[:, 1].min() - .5, X[:, 1].max() + .5
xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.02),
                     np.arange(y_min, y_max, 0.02))

# Predict on the grid points to create a decision boundary
Z = mlp.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)
```

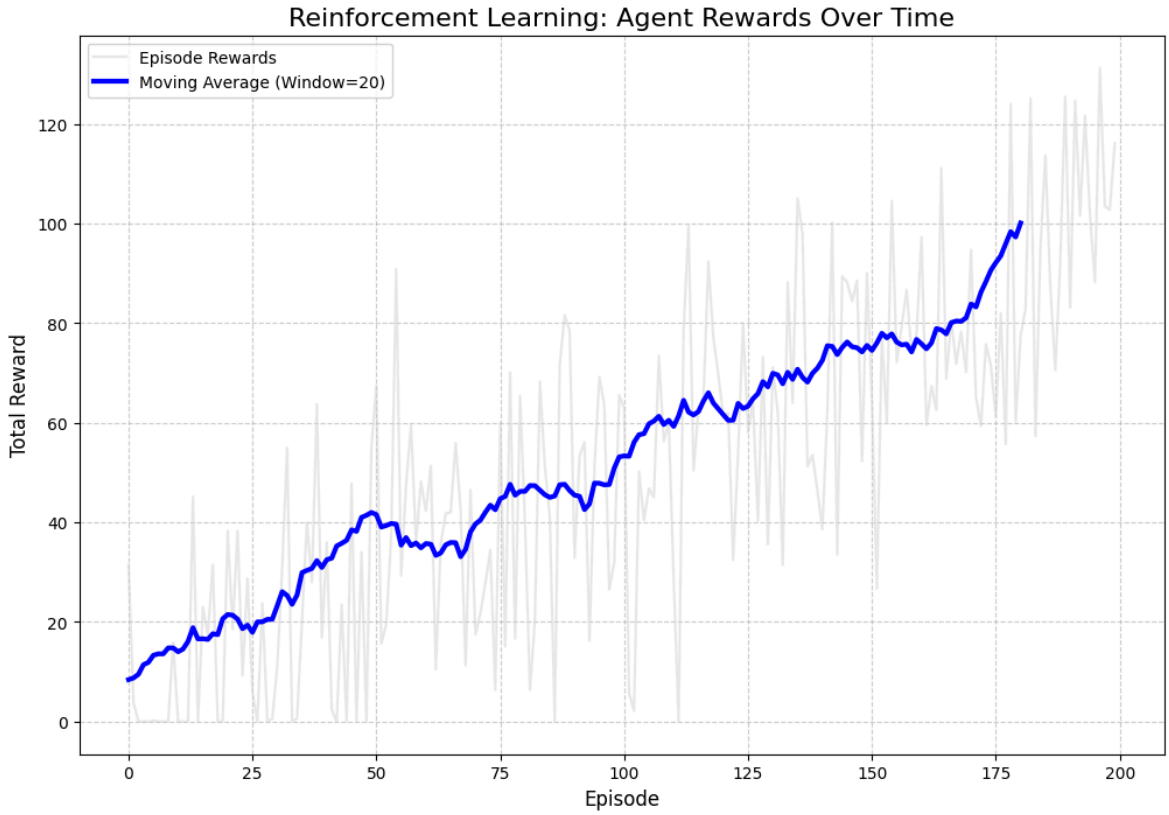
```
# Plot the decision boundary and the data points
plt.contourf(xx, yy, Z, alpha=0.3, cmap=plt.cm.coolwarm)
plt.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap=plt.cm.coolwarm)

# Add title and labels
plt.title('Neural Network Decision Boundary', fontsize=16)
plt.xlabel('Feature 1', fontsize=12)
plt.ylabel('Feature 2', fontsize=12)
plt.grid(True, linestyle='--', alpha=0.6)

plt.show()
```



Reinforcement Learning Rewards Plot Code (Matplotlib)

In [14]:
```
import matplotlib.pyplot as plt
import numpy as np

# 1. Simulate data for rewards over time
num_episodes = 200
# Simulate an agent starting with low rewards and learning over time
rewards = np.linspace(0, 100, num_episodes) + np.random.normal(0, 20, num_episod
rewards[0:50] += np.random.normal(0, 15, 50)  # Add more noise to the beginning
rewards = np.maximum(rewards, 0) # Ensure rewards are non-negative

# 2. Calculate a moving average to smooth the curve
window_size = 20
moving_avg = np.convolve(rewards, np.ones(window_size)/window_size, mode='valid'

# 3. Create the plot
plt.figure(figsize=(12, 8))

# Plot the raw rewards for each episode
plt.plot(rewards, color='lightgray', linestyle='-', alpha=0.5, label='Episode Re
```

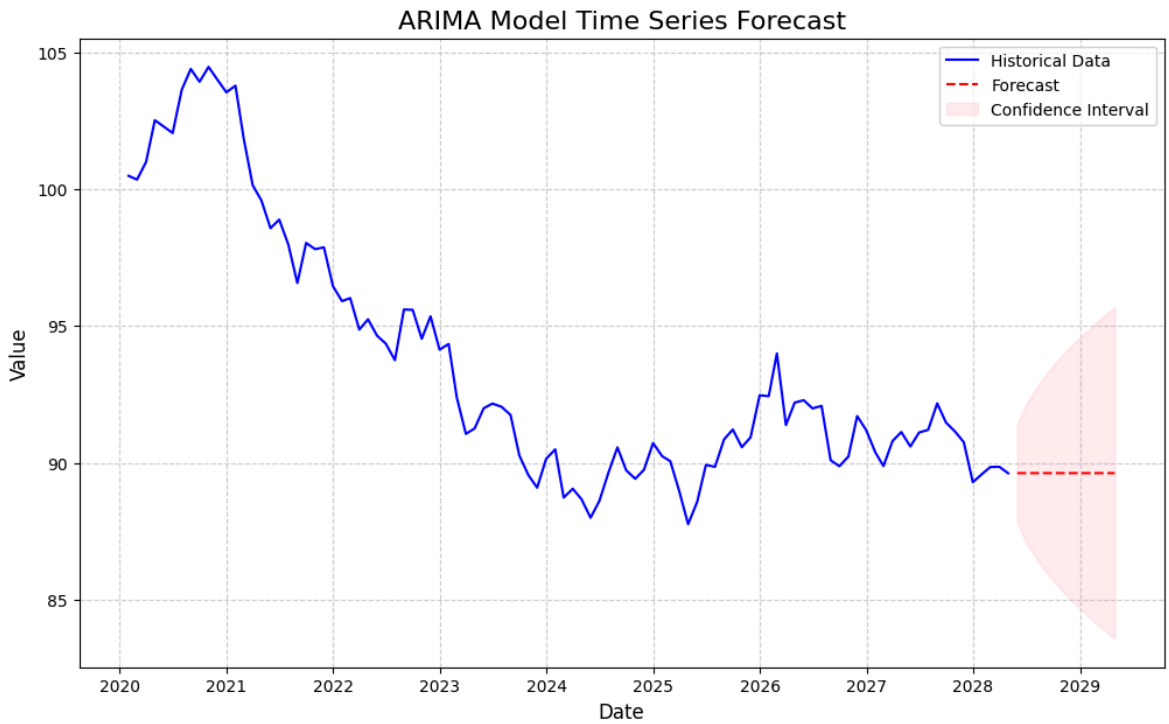
```
# Plot the smoothed moving average
plt.plot(np.arange(len(moving_avg)), moving_avg, color='blue', linewidth=3, labe

# Add titles and labels
plt.title('Reinforcement Learning: Agent Rewards Over Time', fontsize=16)
plt.xlabel('Episode', fontsize=12)
plt.ylabel('Total Reward', fontsize=12)
plt.legend()
plt.grid(True, linestyle='--', alpha=0.6)

plt.show()
```



Time Series Forecasting Chart (ARIMA)

In [12]:
```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from statsmodels.tsa.arima.model import ARIMA

# 1. Create a dummy time series dataset
np.random.seed(42)
dates = pd.date_range(start='2020-01-01', periods=100, freq='ME')  # fixed here
data = np.random.randn(100).cumsum() + 100
ts_data = pd.Series(data, index=dates)

# 2. Fit an ARIMA model
model = ARIMA(ts_data, order=(1, 1, 1))
model_fit = model.fit()

# 3. Get the predictions for the future
forecast_steps = 12
forecast = model_fit.get_forecast(steps=forecast_steps)
```

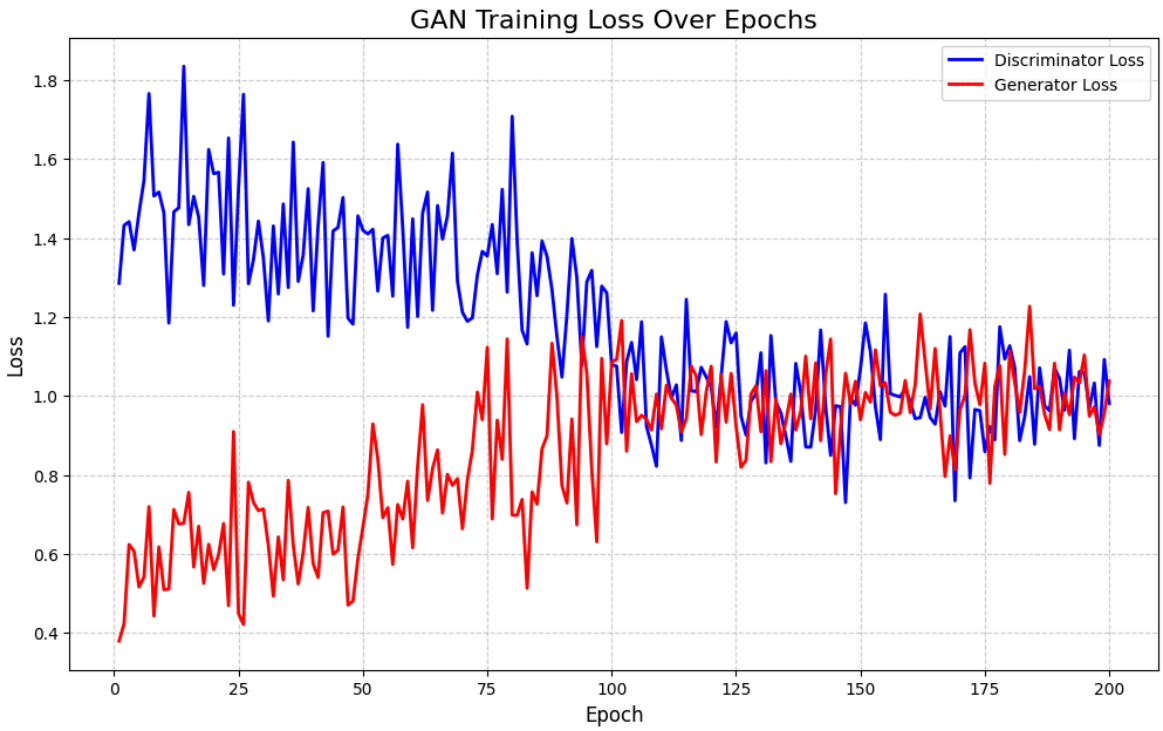
```
forecast_series = forecast.predicted_mean
forecast_ci = forecast.conf_int()

# 4. Create the plot
plt.figure(figsize=(12, 7))

# Plot historical data
plt.plot(ts_data, label='Historical Data', color='blue')

# Plot forecasted data
plt.plot(forecast_series, label='Forecast', color='red', linestyle='--')

# Plot confidence interval
plt.fill_between(forecast_ci.index,
                 forecast_ci.iloc[:, 0],
                 forecast_ci.iloc[:, 1],
                 color='pink', alpha=0.3, label='Confidence Interval')

# Add titles and labels
plt.title('ARIMA Model Time Series Forecast', fontsize=16)
plt.xlabel('Date', fontsize=12)
plt.ylabel('Value', fontsize=12)
plt.legend()
plt.grid(True, linestyle='--', alpha=0.6)

plt.show()
```



Generative Adversarial Network (GAN) Loss Plot

In [18]:
```
import matplotlib.pyplot as plt
import numpy as np

# 1. Simulate the loss data for a GAN during training
epochs = np.arange(1, 201)
```

```
# Discriminator loss starts high and ideally gets lower
disc_loss = 1.5 - (epochs / 400) + np.random.normal(0, 0.15, 200)
# Generator loss starts low, gets high as it gets better, then fluctuates
gen_loss = 0.5 + (epochs / 250) + np.random.normal(0, 0.15, 200)

# Simulate the "truce" where losses balance
gen_loss[100:] = 1.0 + np.random.normal(0, 0.1, 100)
disc_loss[100:] = 1.0 + np.random.normal(0, 0.1, 100)

# 2. Create the plot
plt.figure(figsize=(12, 7))

# Plot the Generator and Discriminator losses
plt.plot(epochs, disc_loss, label='Discriminator Loss', color='blue', linewidth=
plt.plot(epochs, gen_loss, label='Generator Loss', color='red', linewidth=2)

# Add titles and labels
plt.title('GAN Training Loss Over Epochs', fontsize=16)
plt.xlabel('Epoch', fontsize=12)
plt.ylabel('Loss', fontsize=12)
plt.legend()
plt.grid(True, linestyle='--', alpha=0.6)

plt.show()
```

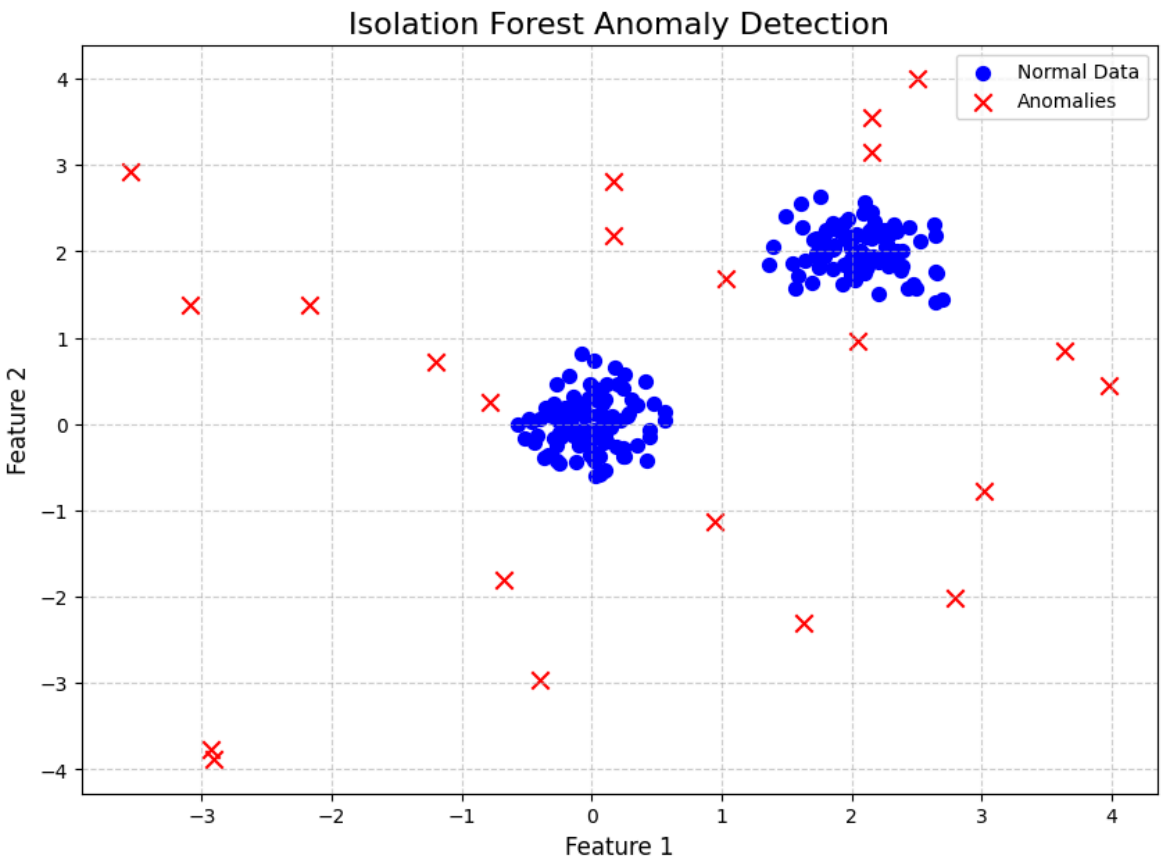


Anomaly Detection Chart (Isolation Forest)

In [19]:

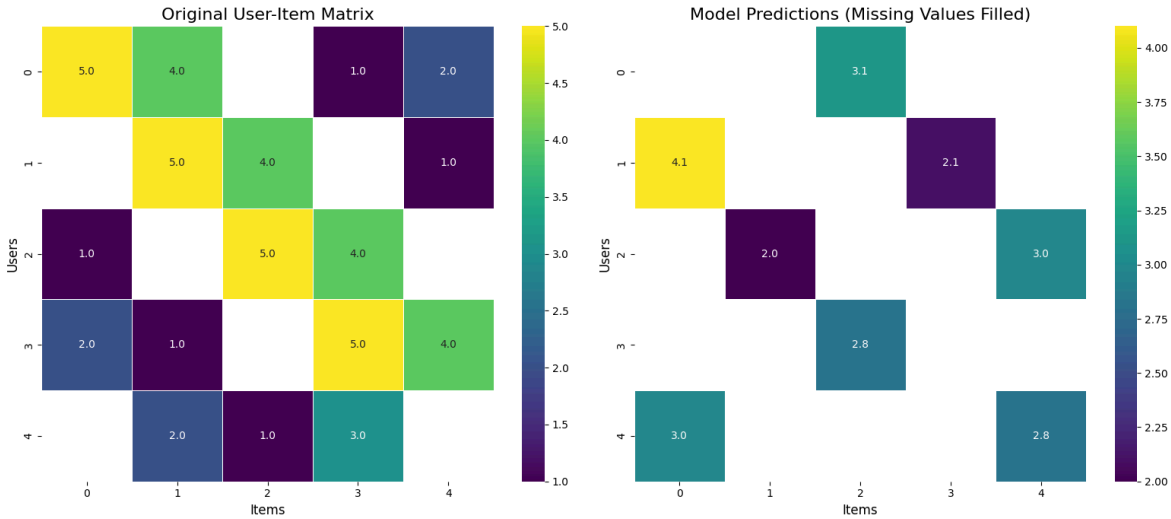
```
import matplotlib.pyplot as plt
import numpy as np
from sklearn.ensemble import IsolationForest

# 1. Create a dummy dataset with some anomalies
rng = np.random.RandomState(42)
X_inliers = 0.3 * rng.randn(100, 2)
```

```
X_inliers = np.concatenate([X_inliers, 0.3 * rng.randn(100, 2) + np.array([2, 2]
X_outliers = rng.uniform(low=-4, high=4, size=(20, 2))
X = np.concatenate([X_inliers, X_outliers], axis=0)

# 2. Fit the Isolation Forest model
model = IsolationForest(contamination=0.1, random_state=42)
model.fit(X)
y_pred = model.predict(X)

# Convert predictions to a more visual format
is_inlier = (y_pred == 1)
is_outlier = (y_pred == -1)

# 3. Create the plot
plt.figure(figsize=(10, 7))

# Plot normal data points
plt.scatter(X[is_inlier, 0], X[is_inlier, 1], s=50, c='blue', label='Normal Data

# Plot identified anomalies
plt.scatter(X[is_outlier, 0], X[is_outlier, 1], s=80, c='red', marker='x', label

# Add titles and labels
plt.title('Isolation Forest Anomaly Detection', fontsize=16)
plt.xlabel('Feature 1', fontsize=12)
plt.ylabel('Feature 2', fontsize=12)
plt.legend()
plt.grid(True, linestyle='--', alpha=0.6)

plt.show()
```



Recommender Systems Chart (Matrix Factorization)

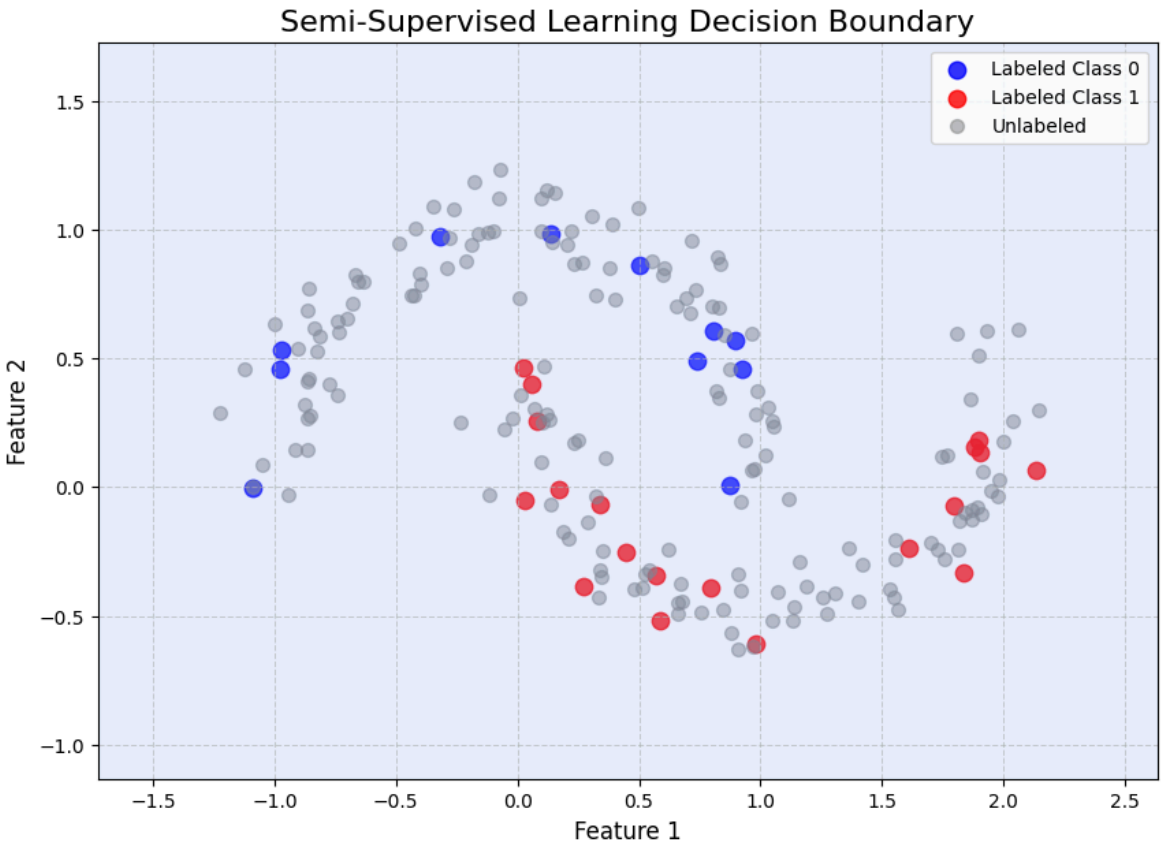
In [2]:
```
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns

# 1. Create a dummy user-item rating matrix with missing values (NaN)
np.random.seed(42)
ratings_matrix = np.array([
    [5, 4, np.nan, 1, 2],
    [np.nan, 5, 4, np.nan, 1],
    [1, np.nan, 5, 4, np.nan],
    [2, 1, np.nan, 5, 4],
    [np.nan, 2, 1, 3, np.nan]
])
# Simulate a filled-in matrix from a matrix factorization model (e.g., SVD)
predicted_matrix = np.array([
    [4.8, 4.2, 3.1, 1.2, 1.8],
    [4.1, 4.9, 4.0, 2.1, 1.1],
    [1.1, 2.0, 4.9, 4.3, 3.0],
    [2.1, 1.2, 2.8, 4.9, 4.2],
    [3.0, 2.2, 1.1, 3.1, 2.8]
])

# Use a mask to only show the original values
mask = np.isnan(ratings_matrix)
predicted_matrix[~mask] = np.nan

# 2. Create the plots
fig, axes = plt.subplots(1, 2, figsize=(16, 7))

# Plot the original ratings matrix
ax1 = sns.heatmap(ratings_matrix, annot=True, cmap='viridis', fmt='.1f', linewid
ax1.set_title('Original User-Item Matrix', fontsize=16)
ax1.set_xlabel('Items', fontsize=12)
ax1.set_ylabel('Users', fontsize=12)

# Plot the predicted ratings (with original values masked)
ax2 = sns.heatmap(predicted_matrix, annot=True, cmap='viridis', fmt='.1f', linew
ax2.set_title('Model Predictions (Missing Values Filled)', fontsize=16)
ax2.set_xlabel('Items', fontsize=12)
ax2.set_ylabel('Users', fontsize=12)

plt.tight_layout()
plt.show()
```



Semi-Supervised Learning Chart

In [3]:
```
import matplotlib.pyplot as plt
import numpy as np
from sklearn.datasets import make_moons
from sklearn.semi_supervised import LabelPropagation

# 1. Create a dummy dataset
X, y = make_moons(n_samples=200, noise=0.1, random_state=42)

# 2. Mark some of the data as unlabeled (-1)
unlabeled_indices = np.random.choice(np.arange(len(y)), size=170, replace=False)
y_labeled = np.copy(y)
y_labeled[unlabeled_indices] = -1

# 3. Fit the semi-supervised model
model = LabelPropagation(kernel='rbf', gamma=0.1, max_iter=1000)
model.fit(X, y_labeled)
y_pred = model.predict(X)

# 4. Create the plot
plt.figure(figsize=(10, 7))

# Plot the labeled and unlabeled data points
plt.scatter(X[y_labeled == 0, 0], X[y_labeled == 0, 1], c='blue', label='Labeled
plt.scatter(X[y_labeled == 1, 0], X[y_labeled == 1, 1], c='red', label='Labeled
plt.scatter(X[y_labeled == -1, 0], X[y_labeled == -1, 1], c='gray', label='Unlab

# Plot the model's decision boundary
x_min, x_max = X[:, 0].min() - .5, X[:, 0].max() + .5
y_min, y_max = X[:, 1].min() - .5, X[:, 1].max() + .5
xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.02), np.arange(y_min, y_max, 0.02
Z = model.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)
plt.contourf(xx, yy, Z, alpha=0.2, cmap=plt.cm.coolwarm)

plt.title('Semi-Supervised Learning Decision Boundary', fontsize=16)
plt.xlabel('Feature 1', fontsize=12)
plt.ylabel('Feature 2', fontsize=12)
plt.legend()
plt.grid(True, linestyle='--', alpha=0.6)
```

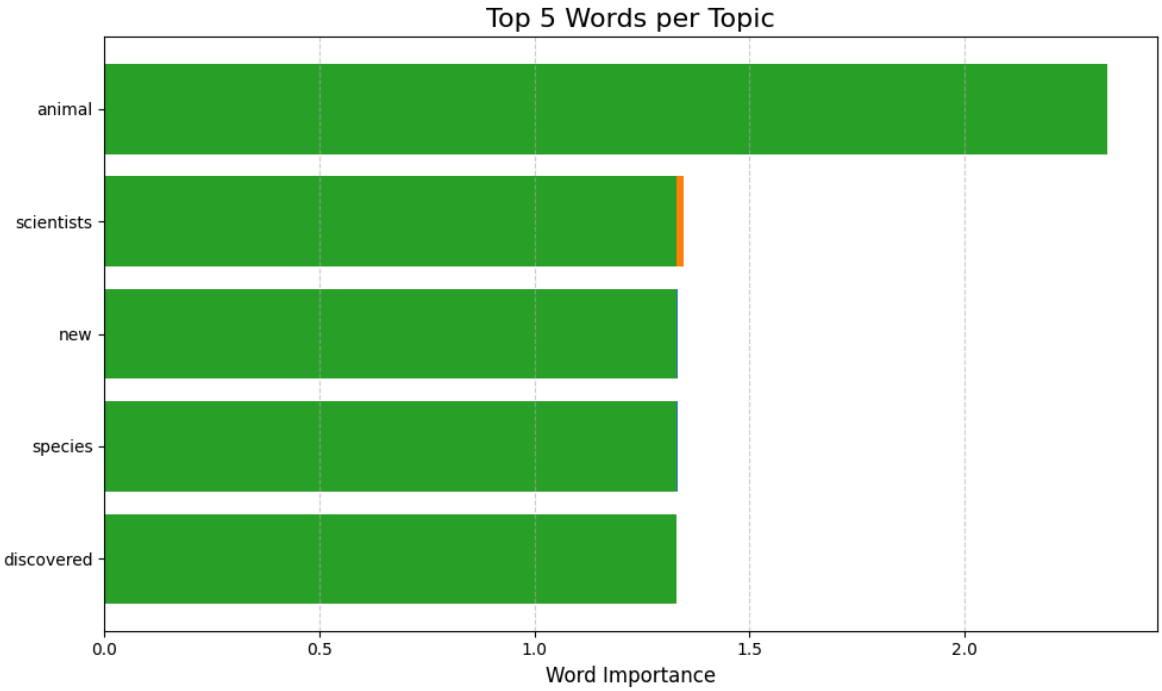
```
plt.show()
```



Topic Modeling Chart (LDA)

In [4]:
```
import matplotlib.pyplot as plt
import numpy as np
from sklearn.decomposition import LatentDirichletAllocation
from sklearn.feature_extraction.text import CountVectorizer

# 1. Create a dummy corpus of documents
documents = [
    "The cat in the hat is a famous story.",
    "A dog chased the cat down the street.",
    "The computer program had a bug and crashed.",
    "The software update fixed the bug in the system.",
    "An elephant is a large wild animal.",
    "A lion is a dangerous wild animal.",
    "Scientists discovered a new species of animal.",
]

# 2. Vectorize the documents
vectorizer = CountVectorizer(stop_words='english')
X = vectorizer.fit_transform(documents)
feature_names = vectorizer.get_feature_names_out()

# 3. Fit the LDA model
lda_model = LatentDirichletAllocation(n_components=3, random_state=42)
lda_model.fit(X)

# 4. Create the plot
def plot_top_words(model, feature_names, n_top_words, title):
```

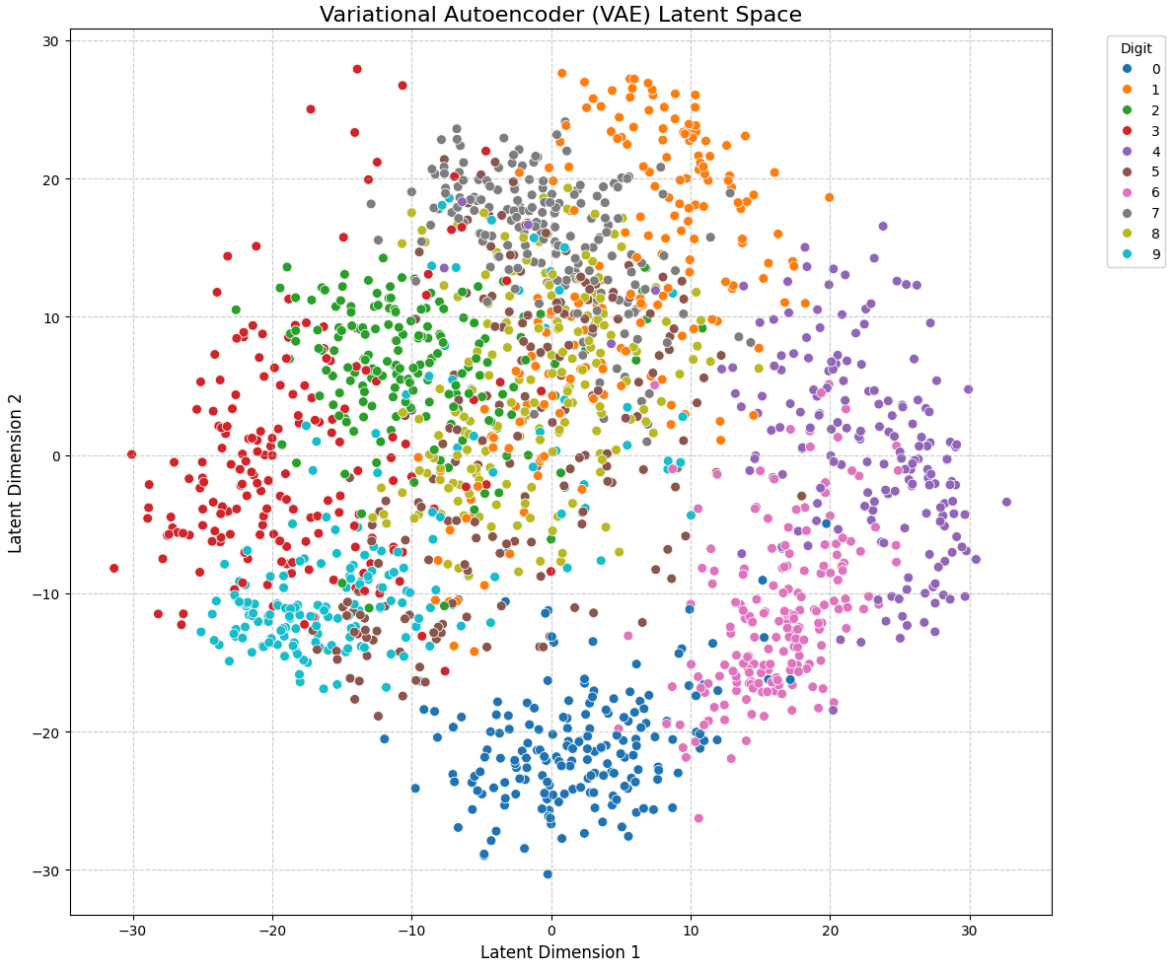
```
    fig, axes = plt.subplots(1, 1, figsize=(10, 6))
    for topic_idx, topic in enumerate(model.components_):
        top_words_idx = topic.argsort()[:-n_top_words - 1:-1]
        top_words = [feature_names[i] for i in top_words_idx]
        weights = topic[top_words_idx]
        axes.barh(np.arange(n_top_words), weights, align='center', label=f'Topic
        axes.set_yticks(np.arange(n_top_words))
        axes.set_yticklabels(top_words)
        axes.invert_yaxis()
        axes.set_title(title, fontsize=16)
        axes.set_xlabel('Word Importance', fontsize=12)
        axes.grid(axis='x', linestyle='--', alpha=0.6)

    plt.tight_layout()
    plt.show()

# Visualize the topics
plot_top_words(lda_model, feature_names, 5, 'Top 5 Words per Topic')
```



Generative Models Chart (Variational Autoencoder - VAE)

In [5]:
```
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
from sklearn.datasets import load_digits
from sklearn.decomposition import PCA

# 1. Load a dataset (e.g., MNIST digits)
digits = load_digits()
X = digits.data
y = digits.target

# 2. Simulate VAE's latent space representation
# In a real VAE, you would get this from the encoder network
np.random.seed(42)
```

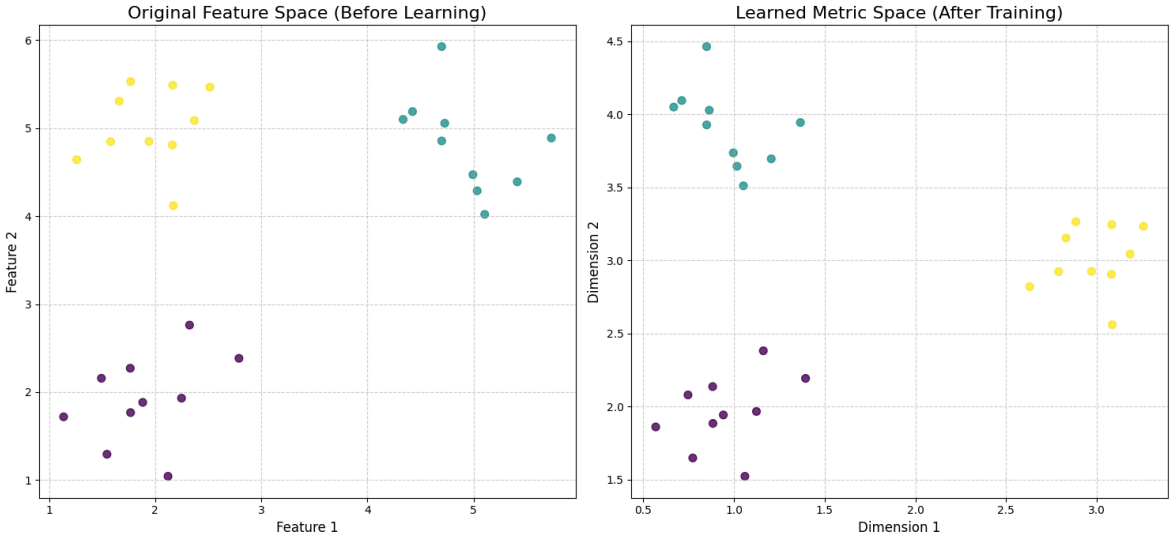
```
latent_space = PCA(n_components=2).fit_transform(X) + np.random.randn(len(X), 2)

# 3. Create the plot
plt.figure(figsize=(12, 10))

# Plot the latent space, colored by the digit class
sns.scatterplot(x=latent_space[:, 0], y=latent_space[:, 1], hue=y, palette='tab1

# Add titles and labels
plt.title('Variational Autoencoder (VAE) Latent Space', fontsize=16)
plt.xlabel('Latent Dimension 1', fontsize=12)
plt.ylabel('Latent Dimension 2', fontsize=12)
plt.grid(True, linestyle='--', alpha=0.6)
plt.legend(title='Digit', bbox_to_anchor=(1.05, 1), loc='upper left')

plt.tight_layout()
plt.show()
```



Metric Learning Chart (Siamese Network)

In [6]:
```
import matplotlib.pyplot as plt
import numpy as np

# 1. Create dummy data with three clusters
np.random.seed(42)
cluster_A = np.random.randn(10, 2) * 0.5 + np.array([2, 2])
cluster_B = np.random.randn(10, 2) * 0.5 + np.array([5, 5])
cluster_C = np.random.randn(10, 2) * 0.5 + np.array([2, 5])
```

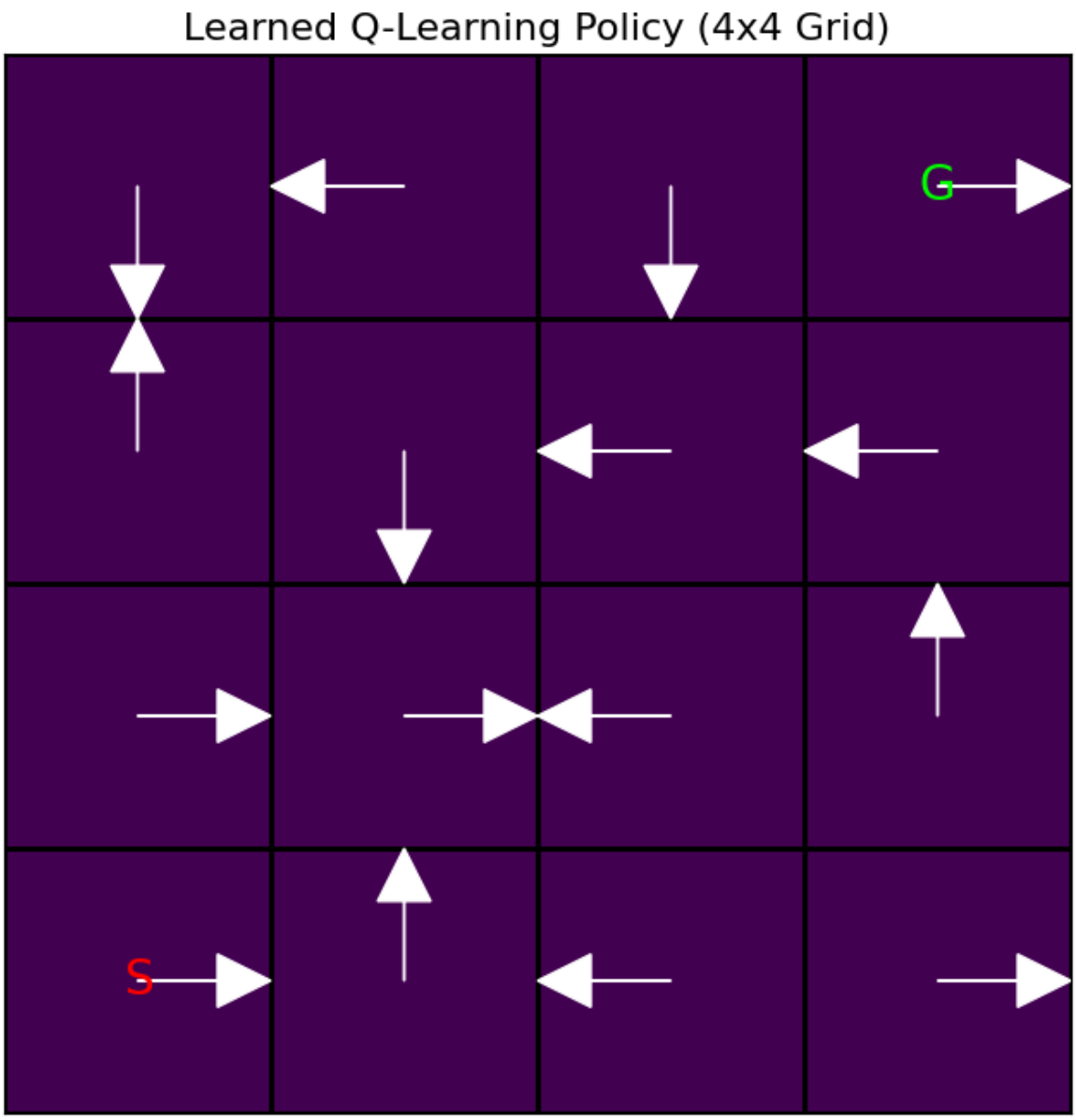
```
# Combine into a single dataset
X = np.concatenate([cluster_A, cluster_B, cluster_C])
y = np.array([0]*10 + [1]*10 + [2]*10)

# 2. Simulate the learned metric space after training
# V_prime represents the learned embedding space
V_prime = np.copy(X)
V_prime[y == 0] = (V_prime[y == 0] - np.array([2,2])) * 0.5 + np.array([1, 2]) #
V_prime[y == 1] = (V_prime[y == 1] - np.array([5,5])) * 0.5 + np.array([1, 4]) #
V_prime[y == 2] = (V_prime[y == 2] - np.array([2,5])) * 0.5 + np.array([3, 3]) #

# 3. Create the plot
fig, axes = plt.subplots(1, 2, figsize=(15, 7))

# Plot the original space
axes[0].scatter(X[:, 0], X[:, 1], c=y, cmap='viridis', s=50, alpha=0.8)
axes[0].set_title('Original Feature Space (Before Learning)', fontsize=16)
axes[0].set_xlabel('Feature 1', fontsize=12)
axes[0].set_ylabel('Feature 2', fontsize=12)
axes[0].grid(True, linestyle='--', alpha=0.6)

# Plot the learned metric space
axes[1].scatter(V_prime[:, 0], V_prime[:, 1], c=y, cmap='viridis', s=50, alpha=0
axes[1].set_title('Learned Metric Space (After Training)', fontsize=16)
axes[1].set_xlabel('Dimension 1', fontsize=12)
axes[1].set_ylabel('Dimension 2', fontsize=12)
axes[1].grid(True, linestyle='--', alpha=0.6)

plt.tight_layout()
plt.show()
```



Reinforcement Learning - Q-Learning Policy Chart

In [8]:
```
import matplotlib.pyplot as plt
import numpy as np

# 1. Simulate a Q-table for 16 states (4x4 grid), 4 actions each
# Actions: 0: up, 1: right, 2: down, 3: left
```

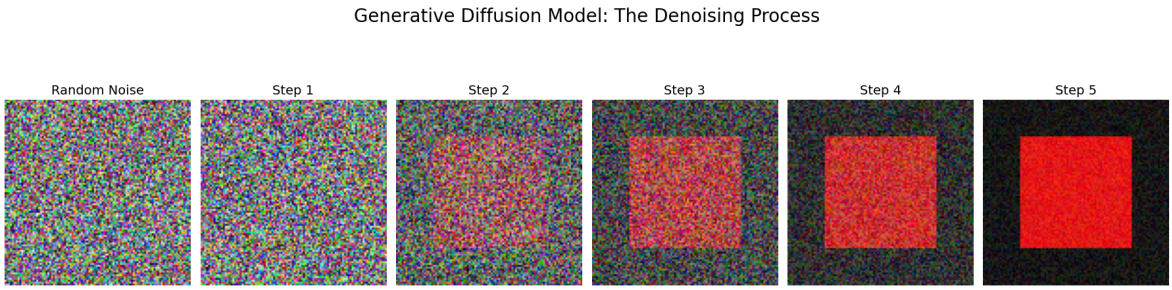
```python
# Q-values are random for demo
np.random.seed(42)
q_table = np.random.randint(1, 10, size=(16, 4))

# 2. Derive the optimal policy (best action per state)
policy = np.argmax(q_table, axis=1).reshape(4, 4)

# 3. Create the plot
plt.figure(figsize=(8, 8))
plt.pcolormesh(np.zeros((4, 4)), edgecolors='black', cmap='viridis')

# Arrow directions for actions
arrows = {0: (0, -1), 1: (1, 0), 2: (0, 1), 3: (-1, 0)}

for i in range(4):
    for j in range(4):
        action = policy[i, j]
        dx, dy = arrows[action]
        plt.arrow(j + 0.5, i + 0.5, dx * 0.3, dy * 0.3,
                  head_width=0.2, head_length=0.2, fc='white', ec='white')

# Add start and goal states
plt.text(0.5, 3.5, 'S', ha='center', va='center', fontsize=20, color='red')
plt.text(3.5, 0.5, 'G', ha='center', va='center', fontsize=20, color='lime')

plt.title('Learned Q-Learning Policy (4x4 Grid)', fontsize=16)
plt.xticks([])
plt.yticks([])
plt.gca().invert_yaxis()
plt.show()
```



Generative Models - Diffusion Process Chart

In [9]:
```
import matplotlib.pyplot as plt
import numpy as np
from PIL import Image

# 1. Simulate the "denoising" process (forward pass of the model)
def generate_images(num_steps):
    images = []
    # Start with pure noise
    img = np.random.rand(100, 100, 3)
    images.append(img.copy())

    # Simulate denoising
    for i in range(num_steps):
        # Add a little bit of the final image and reduce noise
        target_img = np.zeros((100, 100, 3))
        target_img[20:80, 20:80, 0] = 1 # Simple red square target
```

```
        noise_level = (num_steps - i) / num_steps
        img = (1 - noise_level) * target_img + noise_level * np.random.rand(100,
        images.append(img.copy())
    return images

# 2. Get the images for the plot
images = generate_images(num_steps=5)

# 3. Create the plot
fig, axes = plt.subplots(1, 6, figsize=(18, 5))
titles = ['Random Noise'] + [f'Step {i+1}' for i in range(5)]

for i, ax in enumerate(axes.flat):
    ax.imshow(images[i])
    ax.set_title(titles[i], fontsize=14)
    ax.axis('off')

plt.suptitle('Generative Diffusion Model: The Denoising Process', fontsize=20, y
plt.tight_layout()
plt.show()
```

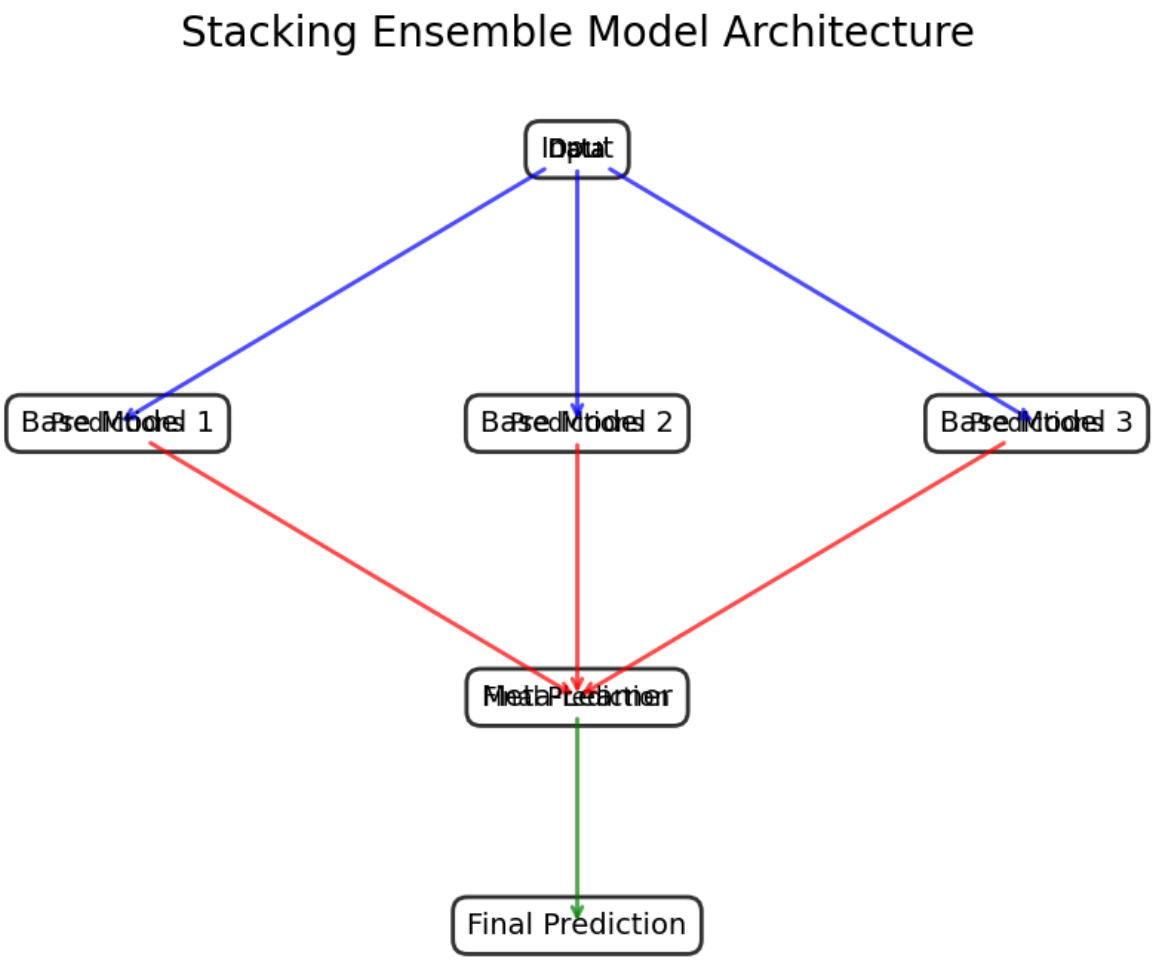


Advanced Ensemble - Stacking Chart

In [11]:

```
import matplotlib.pyplot as plt

# 1. Create a conceptual diagram of the stacking process
fig, ax = plt.subplots(figsize=(10, 8))

# Define node positions and text
nodes = {
    'Input': (0.5, 0.9),
    'Base Model 1': (0.1, 0.6),
    'Base Model 2': (0.5, 0.6),
    'Base Model 3': (0.9, 0.6),
    'Meta-Learner': (0.5, 0.3),
    'Final Prediction': (0.5, 0.05)
}

# Draw boxes for each node
for name, pos in nodes.items():
    ax.text(pos[0], pos[1], name, ha='center', va='center',
            bbox=dict(boxstyle="round,pad=0.5", fc="white", ec="black", lw=2, al
            fontsize=14)

# Fixed arrow drawing function
def draw_arrow(start_node, end_node, text, color='black'):
    ax.annotate(text,
                xy=nodes[end_node],       # arrow points to end node
                xytext=nodes[start_node],# arrow starts from start node
```

```
                arrowprops=dict(arrowstyle='->', lw=2, color=color, alpha=0.7),
                ha='center', va='center', fontsize=12)

# Input to base models
draw_arrow('Input', 'Base Model 1', 'Data', 'blue')
draw_arrow('Input', 'Base Model 2', 'Data', 'blue')
draw_arrow('Input', 'Base Model 3', 'Data', 'blue')

# Base models to meta-learner
draw_arrow('Base Model 1', 'Meta-Learner', 'Predictions', 'red')
draw_arrow('Base Model 2', 'Meta-Learner', 'Predictions', 'red')
draw_arrow('Base Model 3', 'Meta-Learner', 'Predictions', 'red')

# Meta-learner to final prediction
draw_arrow('Meta-Learner', 'Final Prediction', 'Final Prediction', 'green')

# Add title and remove axis
ax.set_title('Stacking Ensemble Model Architecture', fontsize=20)
ax.axis('off')
plt.show()
```



In [ ]: